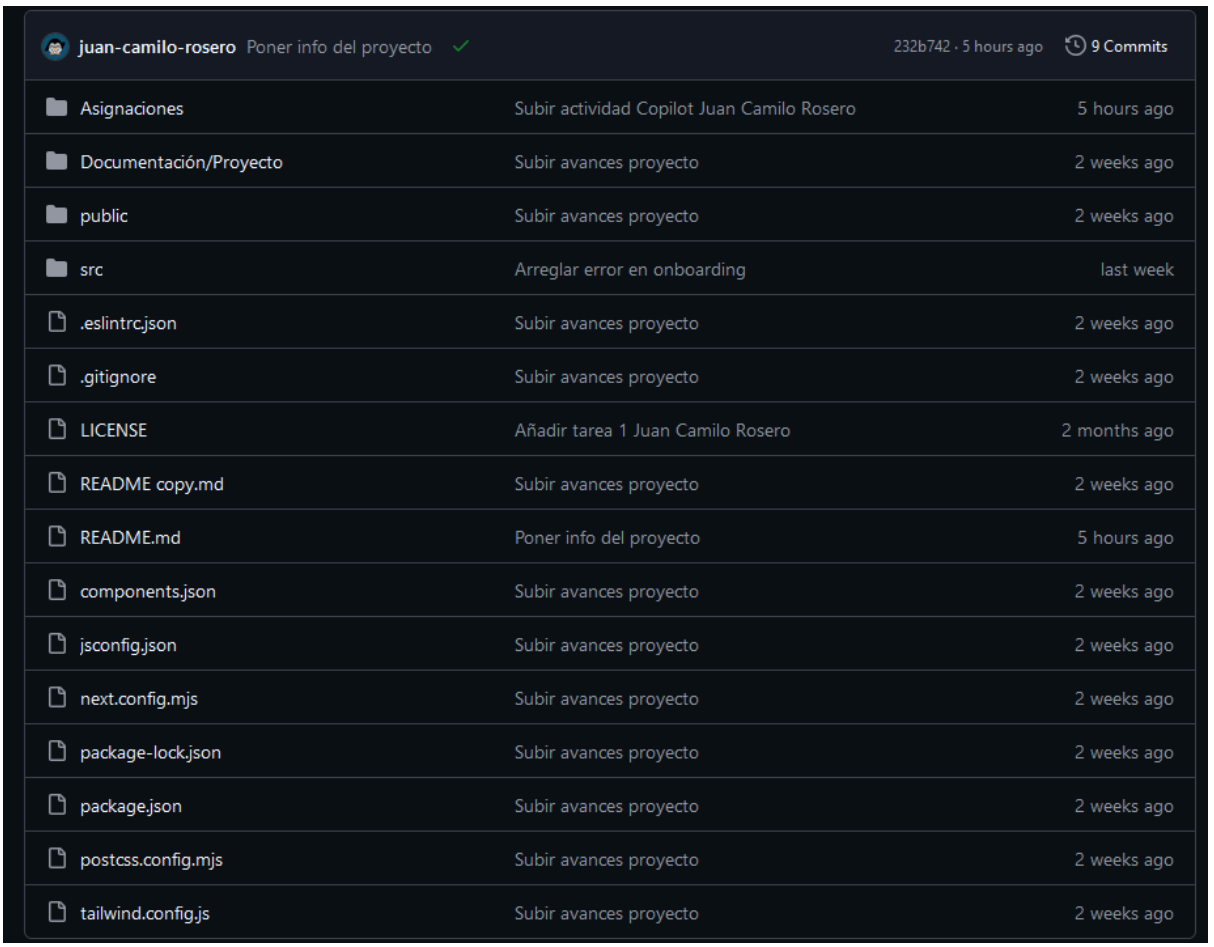
Universidad Nacional de Colombia - Sede Bogotá
Facultad de Ingeniería
Departamento de ingeniería química e ingeniería de sistemas
Curso: Ingeniería de Software

Estudiantes: Marlon David Pabon, Luis Gabriel Peraza, Nicolas Gomez Lozano, Juan Camilo Rosero Santisteban

Proyecto final (los casos de uso e historias de usuario están aquí, nos dimos cuenta de que iban separados hasta las 11)

1. Prerequerimientos:



juan-camilo-rosero Poner info del proyecto ✓ 232b742 · 5 hours ago 9 Commits

Asignaciones	Subir actividad Copilot Juan Camilo Rosero	5 hours ago
Documentación/Proyecto	Subir avances proyecto	2 weeks ago
public	Subir avances proyecto	2 weeks ago
src	Arreglar error en onboarding	last week
.eslintrc.json	Subir avances proyecto	2 weeks ago
.gitignore	Subir avances proyecto	2 weeks ago
LICENSE	Añadir tarea 1 Juan Camilo Rosero	2 months ago
README copy.md	Subir avances proyecto	2 weeks ago
README.md	Poner info del proyecto	5 hours ago
components.json	Subir avances proyecto	2 weeks ago
jsconfig.json	Subir avances proyecto	2 weeks ago
next.config.mjs	Subir avances proyecto	2 weeks ago
package-lock.json	Subir avances proyecto	2 weeks ago
package.json	Subir avances proyecto	2 weeks ago
postcss.config.mjs	Subir avances proyecto	2 weeks ago
tailwind.config.js	Subir avances proyecto	2 weeks ago

README MIT license

Repositorio grupal - Ingeniería de Software 1 - 2024-2 Grupo 2

Integrantes:

- Juan Camilo Rosero
- Nicolás Gómez Lozano
- Luis Gabriel Peraza Muñóz
- Marlon David Pabón Muñóz

Talkie

Talkie es una aplicación web que mediante gamificación busca facilitar el aprendizaje de diferentes dialéctos en varios idiomas. Normalmente las aplicaciones de idiomas enseñan los idiomas de la forma más "neutral" posible, lo cual dificulta muchas veces el entendimiento en entornos con expresiones, palabras y dichos diferentes. Con Talkie buscamos lo contrario: Enseñar idiomas de la forma más específica y personalizada posible.

Tecnologías usadas:

- React
- Next.js
- Tailwind

Servicios externos:

- Firebase
- ChatGPT

2. Levantamiento de requerimientos:

- **¿De dónde surge la idea?**

  Cuando estábamos pensando en la idea del proyecto, uno de los integrantes comentó que estaba creando una IA que podía aprender idiomas y traducir palabras o frases, así que a partir de esa base nos decidimos a crear una app de aprendizaje de idiomas, y analizando las opciones existentes, nos dimos cuenta de que casi siempre se enfocaban en enseñar un idioma de forma “neutral”, pero muy rara vez entran en el terreno de los dialectos, así que decidimos aprovechar ese hueco en el mercado para crear Talkie

- **¿Cómo se pusieron de acuerdo para elegir la idea?**

  Fue bastante fácil, al final de una de las clases nos reunimos un momento, charlamos del tema y vimos que a todos nos parecía la idea de hacer una aplicación de idiomas, de forma que no hizo falta seguir algún proceso más complejo

- **¿Cuáles son las problemáticas que buscan resolver?**

  Buscamos resolver la dificultad de aprender un nuevo idioma y/o dialecto, ya sea por el alto costo de los cursos presenciales, la falta de material en internet o la dedicación necesaria para mantener la motivación.

- **¿Qué expectativas tienen los usuarios potenciales del sistema?**

  Después de un par de semanas o meses de uso, ser capaces de defenderse al hablar en un país con un dialecto que no conocían o del que tenían muy poca información.

- **¿Qué beneficios esperan ustedes al desarrollar este proyecto?**

  Aprender a diseñar y desarrollar un sistema complejo desde cero, trabajando en un equipo de desarrollo, creando la arquitectura, escribiendo el código y haciendo pruebas.
  Siendo algo más específico, queremos también aprender de las tecnologías que usan los otros integrantes, todos programamos con frameworks y lenguajes distintos así que podemos empaparnos un poco de otras áreas del software

3. Análisis de requerimientos:

Must have:

- Inicio de sesión y registro
- Opción de seleccionar idioma y dialecto
- Creación dinámica de lecciones
- Registro de lecciones pendientes y completadas
- Disponibilidad
- Mascota

Should have:

- Diseño atractivo
- Seguridad
- Estadísticas de progreso
- Actividades por cada lección
- Mantenibilidad

Could have:

- Notificaciones
- Modificación de información personal
- Sistema de racha

Won't have:

- Lista de amigos
- Rachas compartidas

Explicaciones:

Inicio de sesión y registro (Mo): Es fundamental para que los usuarios puedan guardar su progreso, acceder a sus lecciones y personalizar su experiencia en la aplicación. Sin este requisito, no habría continuidad en el aprendizaje.

Estimación: 3, si bien es relativamente complejo, ya hemos realizado antes sistemas de autenticación, así que podemos hasta reciclar código de otros proyectos

Opción de seleccionar idioma y dialecto (Mo): Es el eje central del proyecto, ya que su diferenciador es enseñar idiomas con sus variantes dialectales. Sin esta funcionalidad, la aplicación perdería su propósito principal.

Estimación: 13, para que el usuario pueda seleccionar un dialecto y un idioma debe haber detrás toda una base de datos lista para cada uno o alguna conexión con la API de algún modelo de lenguaje, de forma que es de los procesos más demorados y complejos de la aplicación

Creación dinámica de lecciones (Mo): Permite adaptar las necesidades del usuario, asegurando una experiencia de aprendizaje eficiente y personalizada. Sin esto, la app no cumpliría su objetivo educativo ya que el principal diferenciador es la personalización por dialecto.

Estimación: 21, al igual que el requerimiento anterior es uno de los puntos centrales del proyecto, y debe tener detrás bastantes datos ya puestos o una conexión con algún modelo de lenguaje que cree las lecciones (para lo cual

evidentemente deben haber bastantes validaciones y precauciones de seguridad)

Registro de lecciones pendientes y completadas (Mo): Es necesario para hacer un seguimiento del progreso del usuario y mejorar su experiencia de aprendizaje. Sin esta función, el usuario no sabría qué lecciones ha completado ni cuál es su siguiente paso.

Estimación: 5, No es nada del otro mundo, solo necesitamos que el sistema guarde el progreso del usuario y lo muestre bien. Un par de documentos en la base de datos, unas consultas y listo. Suena fácil, pero hay que asegurarse de que no se pierdan datos.

Disponibilidad (Mo): La aplicación debe estar operativa y accesible en todo momento para cumplir su función de aprendizaje continuo. Un sistema inestable haría que los usuarios la abandonen.

Estimación: 8, garantizar la disponibilidad del sistema implica pensar en infraestructura, estabilidad del servidor y manejo de errores. Se deben considerar aspectos como escalabilidad y redundancia para minimizar tiempos de inactividad.

Mascota (Mo): Honestamente siento que sobra la justificación, puede existir Talkie con mascota y sin funcionalidades básicas, pero un Talkie que funcione perfectamente pero sin mascota? N0.

Estimación: 233, Nunca será suficiente tiempo para un proceso tan central.

Diseño atractivo (S): Mejora la experiencia del usuario y lo motiva a seguir usando la aplicación, aunque su ausencia no impediría el aprendizaje.

Estimación: 5, Si bien el diseño no impacta directamente en la funcionalidad, es clave para la experiencia del usuario. Implementarlo requiere tiempo en diseño UI/UX, además de su integración en frontend, pero no es una tarea excesivamente compleja.

Seguridad (S): Protege la información personal del usuario y evita accesos no autorizados. No es imprescindible para el funcionamiento básico, pero sí importante para la confianza de los usuarios.

Estimación: 8, La protección de datos de los usuarios es fundamental para generar confianza, y esto implica implementar cifrado, autenticación segura y protección contra ataques. Su complejidad varía según el nivel de seguridad requerido, por lo que se estima un esfuerzo significativo.

Estadísticas de progreso (S): Ayudan a los usuarios a visualizar su evolución y mantenerse motivados, pero la aplicación podría funcionar sin ellas.

Estimación: 5, Mostrar el avance del usuario requiere una correcta recolección y procesamiento de datos. La implementación de gráficos y análisis de progreso no es excesivamente compleja, pero requiere definir métricas clave y una interfaz clara.

Actividades por cada lección (S): Enriquecen el aprendizaje y refuerzan los conceptos, pero no son estrictamente necesarias para ofrecer contenido educativo.

Estimación: 8, Incluir actividades interactivas implica diseñar distintos tipos de ejercicios, gestionar respuestas y evaluar el rendimiento del usuario. La variabilidad de actividades y su correcta integración con el sistema de lecciones aumentan la complejidad.

Mantenibilidad (S): Facilita la actualización y mejora del sistema a lo largo del tiempo, asegurando su viabilidad a largo plazo.

Estimación: 5, Aunque no es un requerimiento visible para el usuario, una buena arquitectura y documentación facilitan futuras mejoras y correcciones. Su implementación requiere planificación y buenas prácticas de desarrollo, lo que representa un esfuerzo considerable.

Notificaciones (Co): Pueden ayudar a recordar a los usuarios que sigan aprendiendo, pero no son indispensables para el uso de la aplicación.

Estimación: 3, Implementar recordatorios no es complicado, ya que existen servicios de notificaciones push que facilitan su integración. Sin embargo, se debe definir una estrategia para evitar notificaciones excesivas o poco relevantes.

Modificación de información personal (Co): Aporta comodidad al usuario, pero no afecta el aprendizaje si no está presente desde el inicio.

Estimación: 2, Permitir a los usuarios actualizar sus datos es útil, pero su implementación es relativamente sencilla, ya que solo requiere formularios con validaciones y almacenamiento en la base de datos.

Sistema de racha (Co): Refuerza la motivación y la constancia, pero no es imprescindible para el funcionamiento de la aplicación.

Estimación: 5, Para que el sistema de racha funcione correctamente, se debe registrar la actividad del usuario y definir reglas claras sobre cómo se mantiene o pierde una racha. Esto requiere lógica adicional y manejo de fechas, lo que aumenta su complejidad.

Lista de amigos (W): No es esencial para el aprendizaje y requeriría un desarrollo adicional en la parte social, lo cual no es la prioridad en esta versión.

Estimación: N/A, Implica desarrollar una red de usuarios con funciones sociales, lo que desviaría el enfoque de la aplicación.

Rachas entre amigos (W): Aunque puede ser un elemento de motivación, no es una necesidad crítica y puede añadirse en versiones futuras si se demuestra su valor.

Estimación: N/A, Similar al punto anterior, requiere funcionalidades de interacción entre usuarios que no son prioritarias en esta fase.

Consenso final de las estimaciones

Requerimiento	Consenso
Inicio de sesión y registro	3
Opción de seleccionar idioma y dialecto	13
Creación dinámica de lecciones	21
Registro de lecciones pendientes y completadas	5
Disponibilidad	8
Mascota	233
Diseño atractivo	5
Seguridad	8
Estadísticas de progreso	5
Actividades por cada lección	8

Mantenibilidad	5
Notificaciones	3
Modificación de información personal	3
Sistema de racha	5

4. Análisis de gestión de software:

Tiempo:

El proyecto se dividirá en 4 etapas: Diseño de interfaz, desarrollo del material de los cursos, desarrollo de la plataforma y testing. Esto debido a (expande aquí)

El tiempo estimado para el diseño de la interfaz es de aproximadamente dos semanas. Si bien es algo importante, no es indispensable y no hay tanto tiempo. Las pantallas que se diseñarán serán:

- **Landing:** 2 día
- **Onboarding:** 2 días
- **Inicio del dashboard:** 1 día
- **Sección de curso:** 3 días
- **Sección de lección:** 3 días
- **Diseño de alertas y elementos gráficos:** 2 días

El desarrollo del material de los cursos se hará al mismo tiempo por parte de otro integrante del equipo, así que no va a sumar más tiempo en el total. Sin embargo, se puede contar como que cada curso toma 4 días (se harán 3)

Para el proceso de desarrollo de la plataforma, se tomará más tiempo, para un total de mes y medio. Los tiempos se dividirán de la siguiente manera:

- **Setup y Análisis Preliminar:** 5 días
    - Instalación de librerías necesarias e inicialización del proyecto
    - Creación del repositorio
    - Configuración del dominio en Vercel

- **Diseño de la Arquitectura del Sistema:** 2 días
    - Establecimiento de patrones de diseño y estándares de codificación.
    - Elaboración de diagramas de arquitectura
    - Creación del proyecto en Firebase
- **Diseño de la Base de Datos:** 2 días
- **Desarrollo de la aplicación:** 3 semanas
    - Desarrollo de la landing y onboarding: 1 semana
    - Desarrollo del dashboard y conexiones con la BBDD: 1 semana
    - Desarrollo de secciones de lección y curso: 1 semana
- **Deploy y correcciones menores:** 1 semana
    - Comprobar el deploy (en teoría desde el inicio está alojado en Vercel, pero a veces pone errores)
    - Probar responsive y hacer correcciones menores de diseño

Costos:

Los costos principales estarán asociados al uso de Vercel para el alojamiento y Firebase para la autenticación y la base de datos.

- **Vercel:** Vercel ofrece un plan gratuito que incluye 100 GB de transferencia de datos, 1000 funciones de servidor y 1000 minutos de tiempo de ejecución al mes. Esto puede ser suficiente para un proyecto pequeño o en desarrollo.
  Además hay un plan pro que cuesta $20 USD por usuario al mes. Este plan proporciona más capacidades de tráfico, mayor capacidad en el tiempo de ejecución y mejores tiempos de respuesta.

- **Firebase:** Firebase, al igual que Vercel, tiene un modelo basado en el uso, especialmente en lo que respecta a la autenticación, la base de datos en tiempo real (Firestore) y el hosting de funciones.

  Plan gratuito (Spark):

  Autenticación: Permite hasta 10k usuarios activos mensuales de forma gratuita.

Firestore: 1 GB de almacenamiento y 50,000 lecturas, 20,000 escrituras y 20,000 eliminaciones al día de forma gratuita.

Funciones: 2 millones de invocaciones por mes, 400,000 GB-segundos y 200,000 invocaciones de base de datos al mes.

Plan Blaze (de pago por uso):

Autenticación: $0.01 USD por verificación de teléfono (si se usa este método).

Firestore: $0.18 USD por GB almacenado al mes, y $0.06 por cada 100,000 lecturas, $0.18 por cada 100,000 escrituras y eliminaciones.

Funciones: $0.40 por millón de invocaciones, $0.0000004 por GB-segundo.

Estimaciones de costos:

Vercel: Para un proyecto en desarrollo que usa Vercel para el alojamiento y tiene un tráfico moderado, el plan gratuito puede ser adecuado al principio. Si el tráfico crece o se requieren características adicionales (como tiempos de respuesta más rápidos, almacenamiento adicional, etc.), es probable que se necesite un plan Pro.

Costo mensual estimado: Si se usa el plan gratuito al principio, no habrá costos. A medida que el proyecto crece, el costo puede subir a $20 USD/mes por usuario en el plan Pro si se exceden los límites del plan gratuito.

Firebase: Si el proyecto está en la fase inicial con un tráfico moderado, es posible que el plan gratuito de Firebase sea suficiente, especialmente si no se tienen grandes cantidades de datos. Sin embargo, a medida que el tráfico crece, las lecturas y escrituras en la base de datos aumentarán, lo que podría hacer que los costos suban en el plan Blaze.

Costo mensual estimado: Si se utiliza el plan gratuito y no se superan los límites, los costos serían nulos. Si se activa el plan Blaze por las necesidades de base de datos o autenticación, los costos pueden variar dependiendo del uso, pero una estimación razonable podría estar entre $10 a $30 USD mensuales, dependiendo de la cantidad de usuarios y el uso de la base de datos.

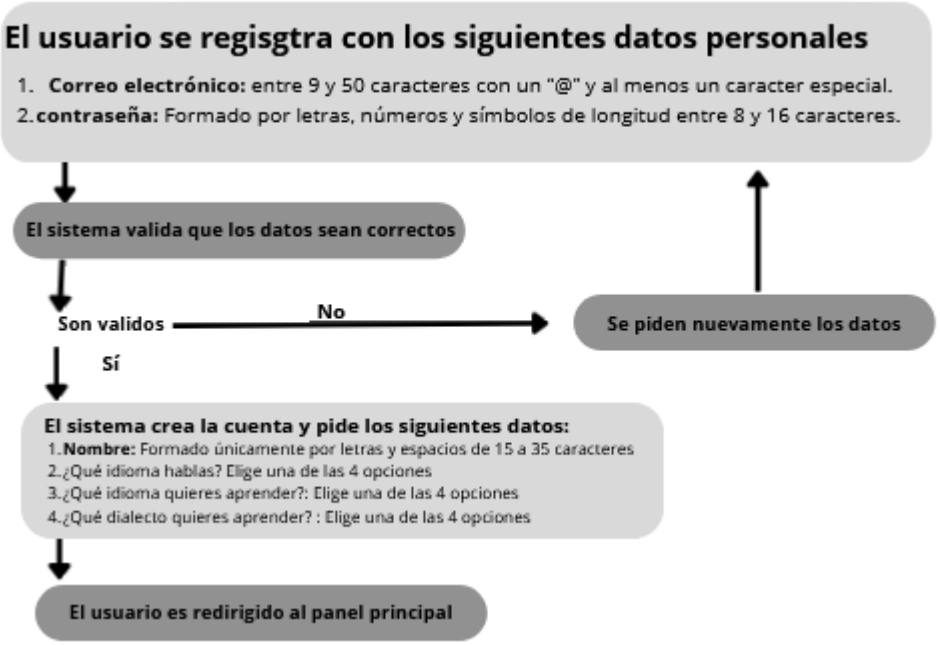
Total estimado: En las primeras fases, utilizando los planes gratuitos de ambas plataformas, los costos podrían ser nulos o muy bajos. Sin embargo, a medida que el proyecto crece, los costos asociados a la escalabilidad y las características avanzadas de ambos servicios pueden ascender a un promedio de $30 a $50 USD/mes para un pequeño proyecto en producción.

Alcance:

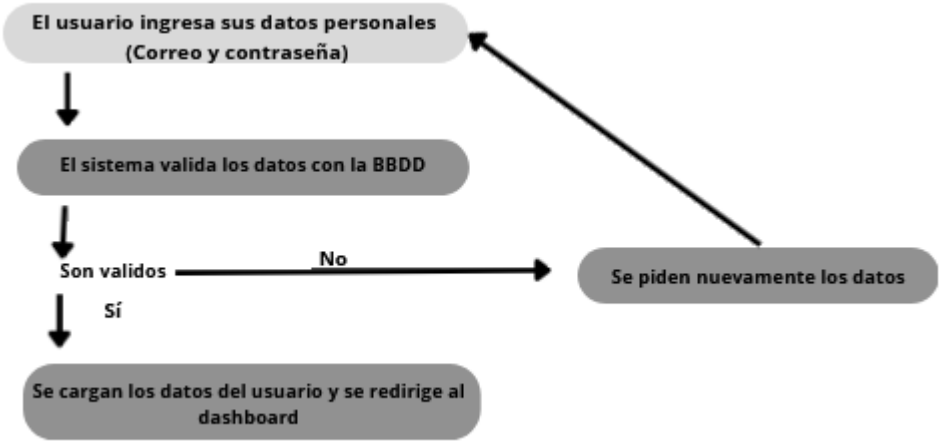
El proyecto de Talkie tiene como objetivo crear una aplicación para aprender idiomas, con un enfoque especial en los dialectos. En la primera versión (MVP), nos vamos a centrar en algunas funciones clave: los usuarios podrán registrarse e iniciar sesión, elegir su idioma y dialecto, crear lecciones de manera dinámica, y llevar un registro de las lecciones que han completado o que están pendientes. También contaremos con una mascota que acompañará al usuario en su proceso de aprendizaje **(es el punto más importante de Talkie)**. Además, nos aseguraremos de que el diseño sea atractivo, que la app sea segura y de que los usuarios puedan tener las actividades de cada lección. La aplicación estará pensada para ser fácil de mantener y actualizar en el futuro. Sin embargo, algunas características como la lista de amigos, las rachas compartidas y las notificaciones quedarán fuera de esta primera versión, ya que nos enfocaremos en lo esencial para que la app cumpla con su propósito principal.

5. Casos de uso del sistema:

Creación de cuenta: Cuando el usuario crea una cuenta y hace el onboarding, se debe validar que la información tenga el formato correcto y dependiendo de eso, dejarlo continuar al dashboard o dejarlo



Inicio de sesión: Cuando el usuario intenta ingresar a su cuenta, se debe validar que los datos ingresados sean correctos



6. Historias de usuario:

URL	Método	Código html
localhost:8080/user	POST	200
Caso de uso técnico Al consultar, debe recibir como dato de entrada el id del usuario y debe retornar código 200 y en data un array con los datos del usuario, 404 en caso de que el usuario no exista		

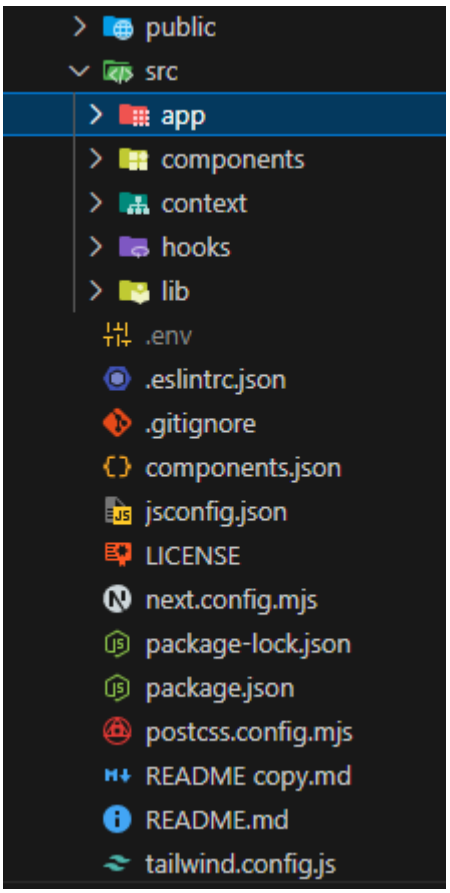
o 402 en caso de que esté mal estructurada la entrada	

Datos de entrada
200:
{
"status": "success",
"data": {
"user": "juanca@gmail.com"
}
}
	Datos de salida
	{
	"status": "success",
	"data": [{
	"name": “Juan Camilo”,
	"language": "español"
	"nativeLanguage": "español”
	“dialect”: “paisa”
	“username”: “juanca”
	}]
	}

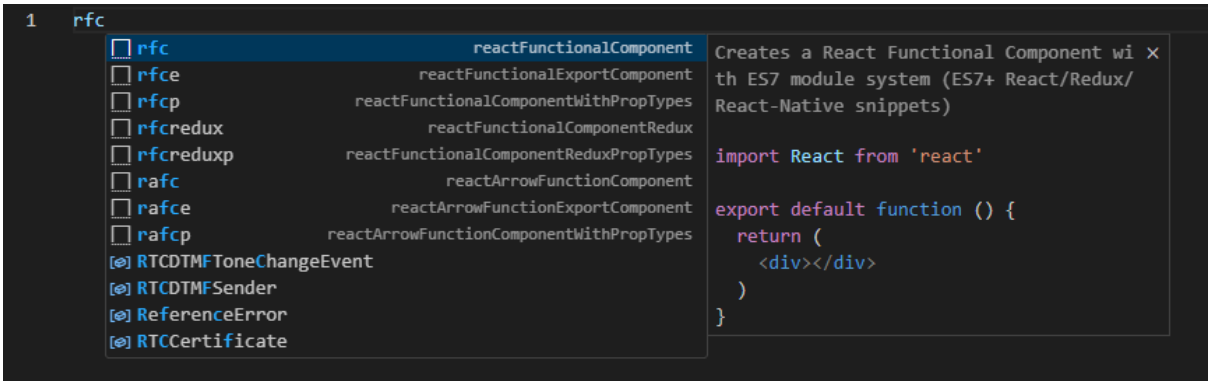
422:
{
"status": "success",
"data": {
"usxer": "juanca@gmail.com"
}
}
	422:
	{
	"status": "unprocessed",
	"message": “No existe el parámetro <usxer>”
	}

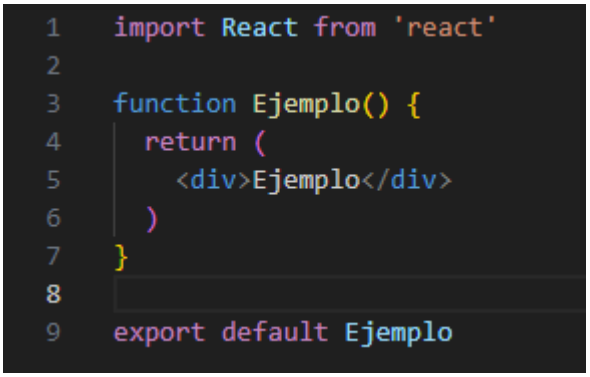
7. Clean code:

Estructura del proyecto:



Implementación de ESlint



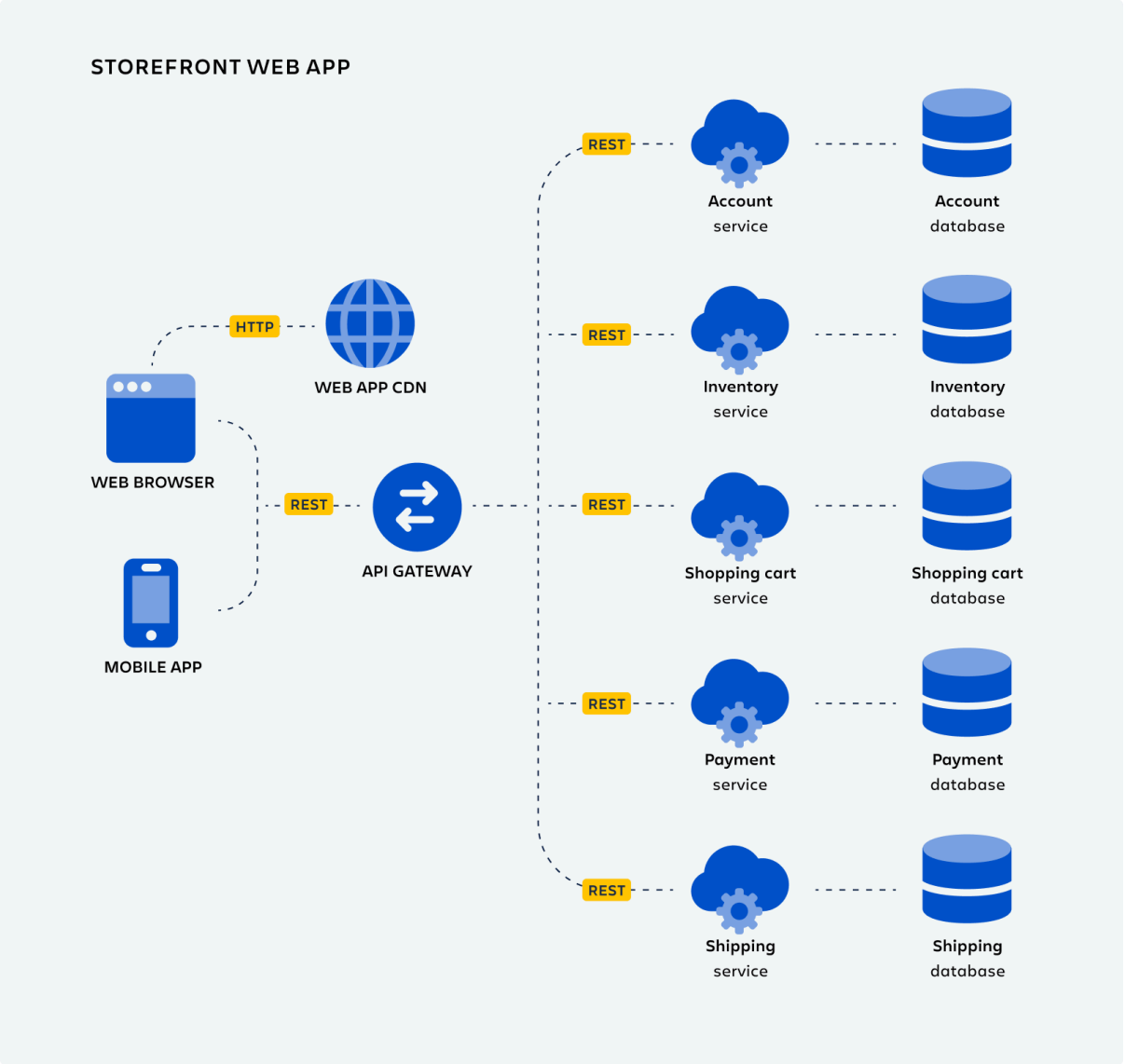


Para el trabajo nos dividimos entre el front y el back para no tener colisiones, de forma que no tuvimos problema entendiendo el código de nuestros compañeros, además de que dividimos los componentes en módulos lo más compactos posibles, por lo que si había código enredado no era tan grave. Y sí se cumple el estándar que designamos :D

8. Diseño y arquitectura:

Arquitectura: Nos decidimos por una arquitectura basada en microservicios principalmente porque parte del backend se gestiona a través de un BaaS (Firebase), lo que facilita la integración y la escalabilidad. Además, esta arquitectura ofrece una gran ventaja en términos de mantenimiento, ya que cada microservicio está compuesto por bloques de código más pequeños y autónomos. Esto no solo mejora la claridad y la simplicidad del sistema, sino que también nos permite incorporar y aprovechar una variedad de servicios externos de manera más eficiente. En resumen, es una opción que facilita la escalabilidad, la flexibilidad y el mantenimiento a largo plazo.

Ejemplo de la arquitectura:



Diseño de BBDD:

Colección Courses:

```
correouwunt@gmail.com
+ Iniciar colección
+ Agregar campo
    completedLessons: 0
  ▸ lessons: [{course: "comidas-rolas",...]
    name: "Comidas Rolas"
    summary: "Este curso te llevará a un viaje culinario por
              Bogotá y la región rola. Conoce los platos
              típicos, postres y bebidas que definen la
              gastronomía de esta tierra."
    totalLessons: 3
    type: "food"
    url: "comidas-rolas"
    video: "https://www.youtube.com/embed/
            fakeComidasIntro"
  ▸ dichos: {completedLessons: 0, less...}
  ▸ expresiones: {completedLessons: 0, less...}(mapa)
```

Colección Users:

```
juanca44@gmail.com
+ Iniciar colección
+ Agregar campo
    dialect: "paisa"
    language: "español"
    name: "Carlos"
    nativeLanguage: "español"
    username: "carlitos 50"
```

9. Patrones de diseño:

Singleton: Lo usamos para el dashboard en React para evitar que hubiese más de 1 sección al mismo tiempo. Mediante este patrón, aseguramos que solo haya una instancia activa del componente de sección en todo momento, lo que mejora el rendimiento y la coherencia en la interfaz de usuario. Cada vez que el usuario navega o cambia de sección, nos aseguramos de que la nueva instancia de la sección reemplace a la anterior sin crear múltiples componentes innecesarios. También se usa para cargar dinámicamente los íconos de los cursos:

```
function Courses() {
  // Función para obtener el icono correcto según la asignatura
  const getIcon = (type) => {
    switch (type) {
      case "food":
        return <RiDrinks2Fill />;
      case "expretions":
        return <RiHeart3Fill />;
      case "people":
        return <RiUser5Fill />;
      case "languages":
        return <RiBookReadLine />;
      default:
        return <RiEarthFill />;
    }
  };
```

Factory Method: Se usa para componentes que tienen varios tipos. En este caso, utilizamos el patrón Factory Method para crear diferentes variantes del botón según el tamaño que se necesite (por ejemplo, sm, md o lg) sin tener que escribir múltiples versiones del componente Button manualmente. Este patrón nos permite centralizar la lógica de creación del botón, lo que facilita la reutilización del código y mejora la flexibilidad.

```jsx
Button.jsx ×
university > ingesoft_I > src > components > general > Button.jsx > Button > bgClass
"use client";

function Button({ text, func, type, size, aditionalStyles = "", disabled = false}) {
  const baseClasses =
    type === "primary"
      ? "text-fgray-200"
      : "text-black border-2 border-fgray-400 hover:border-fgray-600";

  const bgClass = type === "primary" ? "bg-tblue-700 hover:bg-tblue-900" : "bg-transparent";

  const sizeClass = size ? `text-${size}` : "";

  const buttonClasses = `
    font-semibold py-3 md:py-2 px-8 rounded-md outline-none transition-all
    ${baseClasses}
    ${sizeClass}
    ${aditionalStyles} /* Coloca las clases adicionales al final */
    ${bgClass} /* Coloca el fondo predeterminado después para garantizar que no interfiera */
  `.trim();

  return (
    <button onClick={func} className={buttonClasses} disabled={disabled}>
      {text}
    </button>
  );
}

export default Button;
```

State: Este patrón se utiliza mucho para mejorar el rendimiento y facilitar la gestión de componentes que deben cambiar en función de los valores de ciertas variables. Un ejemplo claro de esto es en el caso de las interfaces de usuario que muestran el avance en cursos. En estos casos, la interfaz se carga de manera dinámica, ajustándose y actualizándose en tiempo real cada vez que las variables que almacenan el progreso del curso cambian. De esta forma, se logra una experiencia más fluida y eficiente para el usuario, sin tener que recargar toda la página ni realizar cambios manuales.

```jsx
function LessonsProgress() {
  const [isLoading, setIsLoading] = useState(false);
  const [completedLessons, setCompletedLessons] = useState(0);
  const [precentage, setPrecentage] = useState(0);
  const [totalLessons, setTotalLessons] = useState(0);
```

```
return (
  <div className="w-full bg-fgray-200 p-5 rounded-xl lg:flex lg:flex-col lg:items-center lg:px-0">
    <h2 className="text-2xl font-semibold mb-4 text-center w-full text-fblue-700">
      Próximas lecciones
    </h2>
    <div className="w-full flex flex-col my-4 lg:w-4/5">
      <div className="w-full h-2 bg-fgray-100 rounded-full mb-1 lg:mb-2">
        <div className={`h-1 rounded-full w-[${precentage}%] bg-fblue-700`}></div>
      </div>
      <div className="flex items-center justify-between text-fgray-800 text-sm font-semibold">
        <p>{precentage}% completo</p>
        <p>
          {completedLessons}/{totalLessons} lecciones
        </p>
      </div>
    </div>
    <div className="flex flex-col gap-4 lg:max-h-[50vh] lg:overflow-y-auto py-3 rounded-xl lg:w-4/5">
      {lessons.map((lesson, index) => (
        <LessonProgress lesson={lesson} key={index} preview={true} />
      ))}
    </div>
```